

Rapport du TP4 : Neo4j et langage Cypher

Schéma du graphe (Modèle de données)

Voici le schéma conceptuel du graphe que nous avons modélisé pour ce TP :

```
graph TD
    P1((:Personne\nnom\nage\nville)) -- "[:SUIT]" --> P2((:Personne\nnom\nage\nville))
    P1 -- "[:AIME]" --> I1((:Interet\nnom))
```

(Note: Pour obtenir la capture exacte de votre graphe, exécutez la commande `CALL db.schema.visualization()` dans le navigateur Neo4j <http://localhost:7474> et exportez l'image).

Partie 3 — Recommandation

Requête utilisée :

```
MATCH (alice:Personne {nom: "Alice"})-[:SUIT]->(ami)-[:SUIT]->(suggestion)
WHERE NOT (alice)-[:SUIT]->(suggestion) AND alice <> suggestion
RETURN DISTINCT suggestion.nom, count(ami) AS amis_communs
ORDER BY amis_communs DESC
LIMIT 5
```

Explication de la requête :

- `MATCH (alice)-[:SUIT]->(ami)-[:SUIT]->(suggestion)` : Cette ligne cherche un motif où Alice suit une personne (qui devient `ami`), et cette même personne suit une autre personne (qui devient une `suggestion`). On navigue ainsi à travers les amis d'amis.
- `WHERE NOT (alice)-[:SUIT]->(suggestion) AND alice <> suggestion` : C'est le filtre. On retire toutes les personnes qu'Alice suit déjà (il est inutile de recommander quelqu'un qu'elle suit), et on s'assure de ne pas suggérer Alice à elle-même (dans le cas d'une boucle ou relation réciproque d'un ami).
- `RETURN DISTINCT suggestion.nom, count(ami) AS amis_communs` : On retourne le nom de la recommandation. La fonction `count(ami)` permet de

calculer le nombre de chemins passant par un ami différent (ce qui équivaut au nombre d'amis en commun). 4. `ORDER BY amis_communs DESC LIMIT 5` : On trie les recommandations en plaçant celles ayant le plus d'amis communs en premier (ordre décroissant) et on ne conserve que les 5 meilleures suggestions.

Partie 4 — Réflexion

1. Pourquoi ce type de requête est-il difficile en SQL pur ?

Ce type de requête (recherche de chemins comme les amis d'amis ou les recommandations) nécessite en base de données relationnelle (SQL) de faire des jointures sur la même table de liaison (des *self-joins*). Plus on cherche profondément dans le graphe (ex: amis des amis des amis), plus il faut empiler de jointures. Le coût de ces opérations explose très rapidement, car cela génère des produits cartésiens intermédiaires très coûteux. En outre, la syntaxe SQL devient très complexe et peu lisible, devant parfois faire appel à des clauses récursives (`WITH RECURSIVE`). Contrairement aux SGBD relationnels, Neo4j utilise une approche "index-free adjacency", ce qui signifie que chaque nœud stocke physiquement des pointeurs vers ses voisins. Parcourir les relations (la "traversée" de graphe) est donc une opération à coût constant et extrêmement rapide.

2. Quelle est la limite si le graphe contient 100 millions de nœuds ?

Dans un graphe contenant 100 millions de nœuds et des milliards de relations (de type Facebook ou X/Twitter), l'exécution de cette requête pourrait rencontrer plusieurs limites : *

Les Super-nœuds (dense nodes) : Si un utilisateur (ami d'Alice) suit des millions de personnes, le moteur Cypher va devoir parcourir un nombre massif de relations lors de l'étape `(ami) -[:SUIT]->(suggestion)`, ce qui ralentit considérablement la requête en temps réel. * **L'explosion combinatoire** : Sur les sauts multiples, le nombre de chemins potentiels grandit de façon exponentielle. Une traversée non bornée sur un graphe massif finira par saturer la mémoire et le temps d'exécution. * **La taille de la mémoire (RAM)** : Neo4j tire sa force de sa capacité à mettre en cache les nœuds et relations (PageCache). Sur des graphes massifs de 100 millions de nœuds, un parcours large nécessiterait une RAM colossale pour assurer de bonnes performances, sans quoi on retombe sur des accès disque (I/O) lents. * **Solution** : Dans un vrai grand système, on limite les profondeurs d'exploration de manière stricte, on utilise du pré-calcul (via des processus batch ou des algorithmes du

module GDS - Graph Data Science) plutôt que d'interroger directement en transactionnel (OLTP) pour les recommandations globales.